

✓ CS302 Lab 6: King Nala's Grand Feast

Student Name: Manveer Anand

Student ID: 202351080

Course: CS302 - Distributed and Parallel Computing

Lab: 6

Platform: Google Colab (CUDA C++ / NVCC)

```
# Colab GPU + NVCC check
!nvidia-smi
!nvcc --version
```

✓ Problem Statement

For each candidate key K_c from 1 to 1,000,000, compute satisfaction values for **all** 2^{22} guests, sum them, and compare against target total T .

Formulas:

$$\begin{aligned}\text{raw}[i] &= (37i + 13) \bmod 101 \\ \text{cooked}[i] &= (\text{raw}[i] \cdot K + 59) \bmod 1009 \\ \text{satisfaction}[i] &= \text{cooked}[i] \bmod 101\end{aligned}$$

This notebook uses CUDA C++ kernels and evaluates each candidate key over all guests.

```
%%writefile king_nala_lab6.cu
#include <cuda_runtime.h>
#include <device_launch_parameters.h>

#include <chrono>
#include <stdint>
#include <iostream>
#include <vector>

using ll = long long;

constexpr int N_GUESTS = 1 << 22;
constexpr int K_MAX = 1000000;
constexpr int BLOCK_SIZE = 256;
constexpr int GUEST_BLOCKS = (N_GUESTS + BLOCK_SIZE - 1) / BLOCK_SIZE;
constexpr int K_BATCH = 100;

// Kernel 1: compute satisfaction sums for a batch of candidate keys over all guests
// grid.x -> key index inside current batch
// grid.y -> guest chunk block index
__global__ void computeSatisfactionSum(ll K_base, ll *d_partial)
{
    __shared__ ll smem[BLOCK_SIZE];

    int k_idx = blockIdx.x;
    int gid = blockIdx.y * blockDim.x + threadIdx.x;
    int tid = threadIdx.x;

    ll Kc = K_base + k_idx;
    ll val = 0;

    if (gid < N_GUESTS)
    {
        ll raw = ((ll)gid * 37 + 13) % 101;
        ll cooked = (raw * Kc + 59) % 1009;
        val = cooked % 101;
    }

    smem[tid] = val;
    __syncthreads();

    for (int stride = BLOCK_SIZE / 2; stride > 0; stride >>= 1)
    {
```

```

    if (tid < stride)
        smem[tid] += smem[tid + stride];
    __syncthreads();
}

if (tid == 0)
    d_partial[k_idx * GUEST_BLOCKS + blockIdx.y] = smem[0];
}

// Kernel 2: count unhappy guests (< 40) for one discovered key
__global__ void countUnhappyGuests(ll K, ll *d_partial)
{
    __shared__ ll smem[BLOCK_SIZE];

    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    int tid = threadIdx.x;

    ll val = 0;
    if (gid < N_GUESTS)
    {
        ll raw = ((ll)gid * 37 + 13) % 101;
        ll cooked = (raw * K + 59) % 1009;
        ll sat = cooked % 101;
        val = (sat < 40) ? 1 : 0;
    }

    smem[tid] = val;
    __syncthreads();

    for (int stride = BLOCK_SIZE / 2; stride > 0; stride >>= 1)
    {
        if (tid < stride)
            smem[tid] += smem[tid + stride];
        __syncthreads();
    }

    if (tid == 0)
        d_partial[blockIdx.x] = smem[0];
}

int main()
{
    const int studentId = 202351080;
    const int last4 = studentId % 10000;
    const ll Ktrue = 1 + ((ll)last4 * 101) % 1000000;
    const int G = (last4 % 1000) + 1;

    // Compute target T on CPU using Ktrue
    ll T = 0;
    for (int i = 0; i < N_GUESTS; i++)
    {
        ll raw = ((ll)i * 37 + 13) % 101;
        ll cooked = (raw * Ktrue + 59) % 1009;
        T += cooked % 101;
    }

    ll *d_partial = nullptr, *d_unhappy = nullptr;
    cudaMalloc(&d_partial, (ll)K_BATCH * GUEST_BLOCKS * sizeof(ll));
    cudaMalloc(&d_unhappy, (ll)GUEST_BLOCKS * sizeof(ll));

    std::vector<ll> h_partial((ll)K_BATCH * GUEST_BLOCKS);
    std::vector<ll> h_unhappy(GUEST_BLOCKS);

    ll foundK = -1, cntGreater = 0, cntLess = 0, matchCount = 0;

    auto t_start = std::chrono::high_resolution_clock::now();

    for (ll K_base = 1; K_base <= K_MAX; K_base += K_BATCH)
    {
        int batch = (K_base + K_BATCH - 1 <= K_MAX) ? K_BATCH : (int)(K_MAX - K_base + 1);

        dim3 grid(batch, GUEST_BLOCKS);
        computeSatisfactionSum<<<grid, BLOCK_SIZE>>>(K_base, d_partial);
        cudaMemcpy(h_partial.data(), d_partial, (ll)batch * GUEST_BLOCKS * sizeof(ll), cudaMemcpyDeviceToHost);

        for (int k = 0; k < batch; k++)
        {

```

```

11 total = 0;
for (int b = 0; b < GUEST_BLOCKS; b++)
    total += h_partial[(11)k * GUEST_BLOCKS + b];

11 Kc = K_base + k;
if (total == T)
{
    if (foundK == -1)
        foundK = Kc;
    matchCount++;
}
else if (total > T)
    cntGreater++;
else
    cntLess++;
}
}

auto t_end = std::chrono::high_resolution_clock::now();
double elapsed = std::chrono::duration<double>(t_end - t_start).count();

11 Kuse = (foundK != -1) ? foundK : Ktrue;

11 rawG = ((11)G * 37 + 13) % 101;
11 cookedG = (rawG * Kuse + 59) % 1009;
11 satG = cookedG % 101;

// Run Kernel 2 for unhappy guest count using discovered key
countUnhappyGuests<<<GUEST_BLOCKS, BLOCK_SIZE>>>(Kuse, d_unhappy);
cudaMemcpy(h_unhappy.data(), d_unhappy, (11)GUEST_BLOCKS * sizeof(11), cudaMemcpyDeviceToHost);

11 unhappy = 0;
for (int b = 0; b < GUEST_BLOCKS; b++)
    unhappy += h_unhappy[b];

std::cout << "Assignment summary\n";
std::cout << "-----\n";
std::cout << "Student ID           : " << studentId << "\n";
std::cout << "Last 4 digits           : " << last4 << "\n";
std::cout << "Ktrue                   : " << Ktrue << "\n";
std::cout << "Target total T         : " << T << "\n";
std::cout << "K found by ascending search : " << foundK << "\n";
std::cout << "Count(total < T)       : " << cntLess << "\n";
std::cout << "Count(total > T)       : " << cntGreater << "\n";
std::cout << "Total matching keys    : " << matchCount << "\n";
std::cout << "Guest number G         : " << G << "\n";
std::cout << "raw[G]                 : " << rawG << "\n";
std::cout << "cooked[G] using K      : " << cookedG << "\n";
std::cout << "satisfaction[G] using K : " << satG << "\n";
std::cout << "Guest kernel blocks    : " << GUEST_BLOCKS << "\n";
std::cout << "Guest kernel threads/block : " << BLOCK_SIZE << "\n";
std::cout << "Total launched threads : " << (11LL * GUEST_BLOCKS * BLOCK_SIZE) << "\n";
std::cout << "Unhappy guests (<40)   : " << unhappy << "\n";
std::cout << "Execution time         : " << elapsed << " seconds\n";

cudaFree(d_partial);
cudaFree(d_unhappy);
return 0;
}

```

Writing king_nala_lab6.cu

✓ CUDA C++ Implementation

The next cell writes the CUDA C++ source file with required kernels:

- `computeSatisfactionSum` (batch over candidate keys, all guests)
- `countUnhappyGuests` (for discovered key)

Then compile with `nvcc` and run.

```

# Compile the CUDA C++ program
!nvcc -O2 king_nala_lab6.cu -o king_nala_lab6

```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future rele
```

```
# Run the CUDA C++ executable
!./king_nala_lab6
```

Assignment summary

```
-----
Student ID           : 202351080
Last 4 digits        : 1080
Ktrue                : 109081
Target total T       : 209590540
K found by ascending search : 109
Count(total < T)      : 480672
Count(total > T)      : 518337
Total matching keys   : 991
Guest number G        : 81
raw[G]                : 81
cooked[G] using K     : 816
satisfaction[G] using K : 8
Guest kernel blocks   : 16384
Guest kernel threads/block : 256
Total launched threads : 4194304
Unhappy guests (<40)   : 1661109
Execution time         : 203.495 seconds
```